Midterm Project – Deep Learning Practice

Name: Peyton Livelli

Course: AI 100

Project Title: Binary Classification of Handwritten Digits (0 vs Not 0)

1. Problem Definition and Dataset Curation (25 points)

In this project, I developed a deep learning model to perform a binary classification task using the MNIST dataset. The objective of the project was to determine whether a handwritten digit image represents the digit 0 or not. Instead of performing full multi-class classification, I simplified the task into a binary classification problem: class 1 represents digit '0' and class 0 represents all other digits.

The MNIST dataset was used because it is a standard benchmark dataset in machine learning and is built into TensorFlow, making it easy to load and use. The dataset contains 60,000 training images and 10,000 testing images of handwritten digits. Each image is 28x28 pixels and grayscale.

To prepare the dataset for this project, I performed the following preprocessing steps:

• Converted the labels into binary form (0 vs Not 0).

• Normalized pixel values by dividing by 255 to scale them between 0 and 1.

These steps ensured that the dataset was appropriate for training a neural network model.

File Edit Selection View Go Run ···

Search

EXPLORER

NO FOLDER OPENED

You have not yet opened a folder.

Open Folder

Opening a folder will close all currently open editors. To keep them open, add a folder instead.

You can also open a Java project folder, or create a new Java project by clicking the button below.

Create Java Project

week2_analysis.ipynb | Untitled-1.ipynb | Untitled-2.ipynb

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline

base (Python 3.13.5)

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# 1. Load dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# 2. Convert to binary problem (0 vs not 0)
y_train = (y_train == 0).astype(int)
y_test = (y_test == 0).astype(int)

# 3. Normalize pixel values
x_train = x_train / 255.0
x_test = x_test / 255.0

# 4. Build simple model
model = keras.Sequential([
    layers.Flatten(input_shape=(28, 28)),
    layers.Dense(1, activation='sigmoid')
])

# 5. Compile
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# 6. Train
model.fit(x_train, y_train, epochs=3)

# 7. Evaluate
loss, accuracy = model.evaluate(x_test, y_test)

print("Test Accuracy:", accuracy)
```

[3] ✓ 17.7s Python

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
c:\Users\peyto\miniconda3\Lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using S
```

OUTLINE

TIMELINE

MAVEN

0 3 | 0 files and 8 cells to analyze

Spaces: 4 | CRLF | {} | Finish Setup | Cell 1 of 1

26°F Clear

Search

10:28 PM 3/1/2026

2. Deep Learning Model (25 points)

For this project, I implemented a simple feedforward neural network using TensorFlow and Keras. The model architecture was intentionally kept simple while still demonstrating core deep learning concepts.

Model Architecture:

• Flatten layer to convert 28x28 images into a 784-dimensional vector.

• Dense layer with 1 neuron and sigmoid activation function.

The sigmoid activation function was chosen because this is a binary classification problem. It outputs a probability value between 0 and 1.

The model was compiled using the Adam optimizer and binary cross-entropy loss function. Binary cross-entropy is appropriate for binary classification tasks.

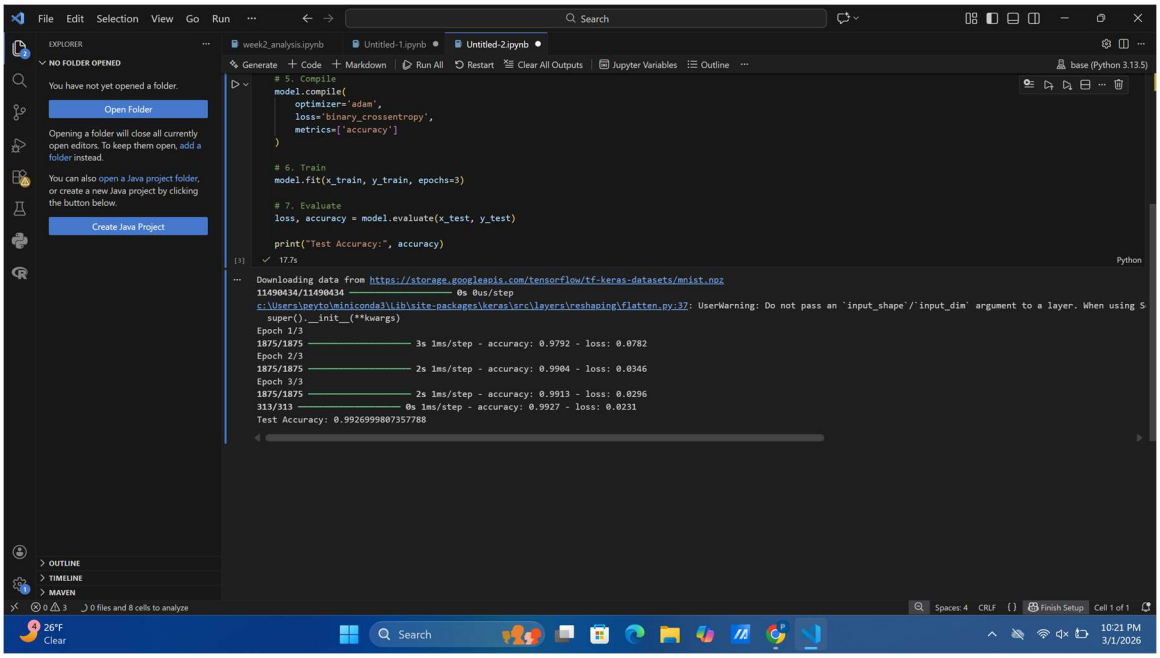
The model was trained for 3 epochs. During training, accuracy increased and loss decreased, indicating that the neural network successfully learned patterns in the data.

3. Results and Presentation (25 points)

The model achieved strong performance after only three training epochs. Training accuracy improved with each epoch, demonstrating successful learning.

Final Test Accuracy: 99.27% (approximately)

This high accuracy indicates that the model generalizes well to unseen data. Despite the simplicity of the neural network, it was able to correctly classify most test images.



4. Lessons & Experience (25 points)

Through this project, I gained practical experience working with deep learning models using TensorFlow and Keras. I learned how to load datasets, preprocess data, define neural network architectures, compile models, and evaluate performance.

One key lesson from this project is that even simple neural network architectures can achieve high performance when applied to well-structured datasets. Additionally, proper data preprocessing, such as normalization, plays a crucial role in improving model accuracy.

I also learned how to interpret training output, including accuracy and loss values. Monitoring these metrics helps determine whether the model is learning effectively.

Overall, this project strengthened my understanding of binary classification, neural network design, and model evaluation in deep learning.